AI-Powered Symptom Triage & Health Guidance Assistant (Non-Diagnostic)

Name : Willy Cornelius

Department : Information System

Institution : Tarumanegara University

ABSTRACT

This project focuses on developing an AI-powered chatbot that provides general health guidance based on user-reported symptoms. The system is designed to ensure safe, structured, and non-diagnostic responses using prompt engineering techniques.

The application integrates a Large Language Model (LLM) through API and presents the results through a chatbot interface built with Streamlit. The system emphasizes safety, clarity, and responsible AI usage in a healthcare-related context.

OBJECTIVE

The main objectives of this project are:

- To build an AI chatbot that provides non-diagnostic health guidance
- To apply structured prompt engineering (RTCFC, persona, constraints)
- To ensure safe and controlled AI responses
- To develop a functional web-based chatbot application

TECHNOLOGY USED

- Python
- Streamlit
- OpenRouter API (LLM Integration)

- Prompt Engineering
- JSON

SYSTEM ARCHITECTURE

The system workflow can be described as:

User Input → Prompt Builder → LLM API → AI Response → UI Display

- User inputs symptoms
- System builds structured prompt
- Prompt is sent to API
- AI generates response
- Response is displayed

DETAILED PROJECT EXECUTION

1. Environment Setup

The development environment is set up using Python and required libraries such as Streamlit and requests.

2. Prompt Design (Core System)

The system uses structured prompts to control AI behavior:

- Role definition (non-diagnostic assistant)
- Task instruction (analyze symptoms)
- Output format (structured response)
- Constraints (safety rules)

This ensures that the AI produces consistent and safe responses.

3. API Integration

The system integrates with OpenRouter API to communicate with the AI model.

- The prompt is sent using JSON format
- The API processes the request

- The response is returned and displayed

4. Chatbot Implementation

The chatbot is implemented using Streamlit with features such as:

- Chat input interface
- Conversation history
- Multi-turn interaction

The system also stores previous messages to improve response relevance.

5. Context Handling

The system includes previous conversation context to improve AI responses.

This allows the AI to:

- understand follow-up questions
- adjust urgency level
- provide more relevant output

PROJECT DEMONSTRATION : A Real-World Example

Step 1: User Input

A user opens the application through the web interface and enters their symptoms into the chatbot.

For example:

“I feel dizzy”

At this stage, the system captures the input and prepares it for processing.

Step 2: Prompt Construction and Processing

Once the input is received, the system constructs a structured prompt using predefined components such as role, task, format, and constraints.

The prompt ensures that:

- The AI behaves as a non-diagnostic assistant

- The response follows a structured format
- Safety rules are enforced

This prompt is then sent to the LLM API for processing.

Step 3: AI Response Generation

The AI processes the prompt and generates a structured response consisting of:

- Possible Causes (e.g., dehydration, fatigue)
- Recommended Actions (e.g., rest, hydration)
- Urgency Level (based on symptom severity)
- Disclaimer (ensuring non-diagnostic guidance)

The response is then displayed to the user through the chatbot interface

4. Follow-Up Interaction (Multi-Turn Conversation)

The user continues the conversation by providing additional information.

For example:

“I’ve been feeling it for 2 days”

At this stage, the system utilizes previous conversation context to improve the response.

6. **System Behavior and Safety Enforcement**

Throughout the interaction, the system ensures:

- No medical diagnosis is provided
- No medication is prescribed
- All responses remain general and cautious
- A clear disclaimer is always included

This guarantees that the system remains safe and aligned with responsible AI.

DOCUMENTATION CAPSTONE (SCREENSHOTS)

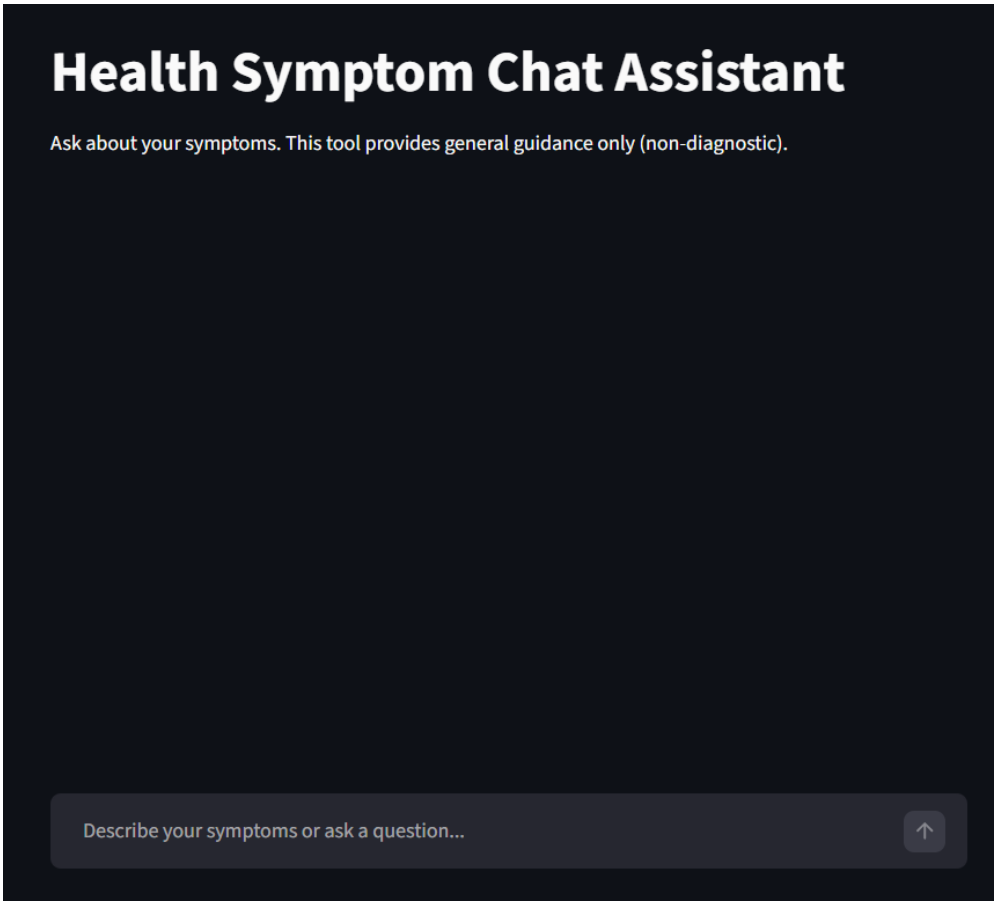

Fig: Initial Interface of Health Symptom Chat Assistant

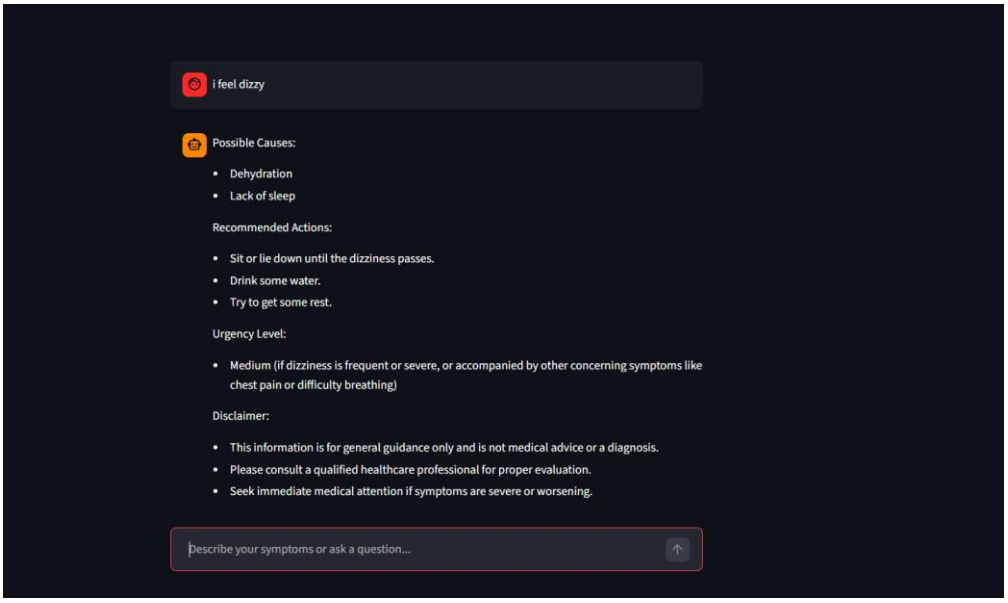

Fig: AI-Generated Health Guidance Output

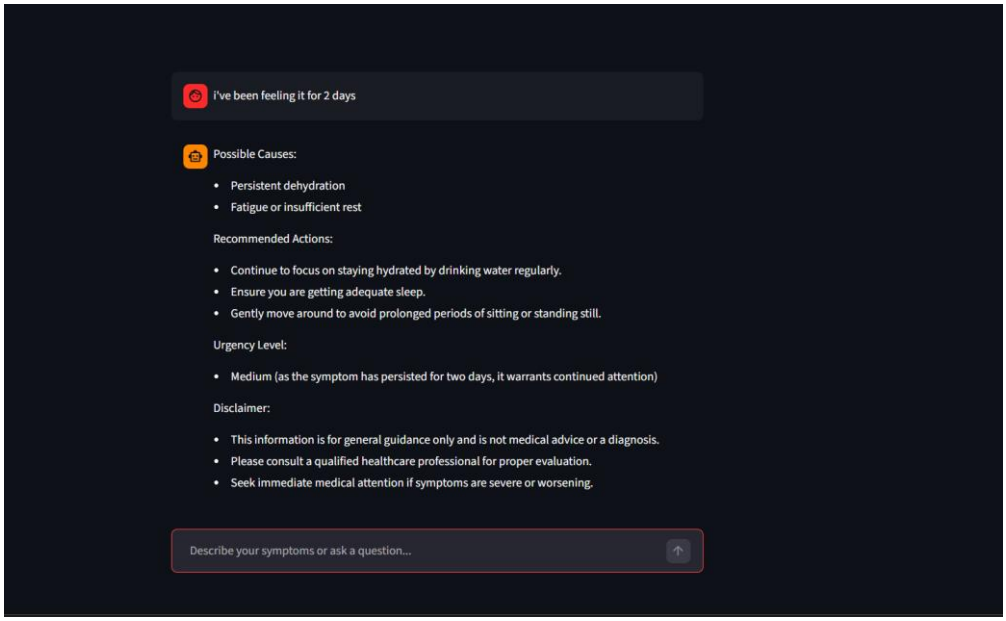

Fig: Multi-turn Conversation with Context Handling

CONCLUSION

This project demonstrates the development of an AI-powered health guidance assistant that provides structured, non-diagnostic responses based on user-reported symptoms. By applying prompt engineering techniques such as RTCFC, persona design, and safety constraints, the system ensures consistent and responsible AI behavior. The integration of a chatbot interface using Streamlit enables intuitive user interaction, while context handling improves response relevance in multi-turn conversations. Importantly, the system prioritizes safety by avoiding diagnosis and including clear disclaimers. Overall, this project highlights how AI can be effectively utilized in sensitive domains while maintaining ethical boundaries, user safety, and reliable output through controlled prompt design.

FUTURE SCOPE

This project can be further enhanced through several improvements to increase its usability, scalability, and overall performance.

1. Deploy the Application Online: The application can be deployed to a cloud-based platform so that it is accessible beyond a local environment. By hosting the system online, users can interact with the chatbot anytime and from any location without requiring local setup. This improvement would significantly enhance the practicality and real-world usability of the system.

2. Improve Long-Term Memory Handling : Currently, the system only utilizes a limited number of recent messages as context. Future development can include enhanced memory mechanisms that allow the chatbot to retain and recall longer conversation histories. This would result in more consistent, personalized, and context-aware interactions over extended sessions.

3. Enhance UI Design: The user interface can be further improved to provide a more engaging and intuitive experience. Enhancements such as better layout design, improved chat styling, and more interactive elements would make the system more user-friendly and visually appealing, increasing overall user satisfaction.

4. Add Multilingual Support: The system can be extended to support multiple languages, allowing users from different linguistic backgrounds to interact with the chatbot. This would broaden the accessibility of the application and make it more inclusive for a wider range of users.